

An Empirical Study of Layer Normalization in DDPG and TD3

Martin Delamare, Youssef Hamzaoui

October 9, 2026

1 Introduction

DDPG uses a critic network to estimate how good an action is in a given state. Since these estimates are not exact, some actions may appear better than they really are. The actor can then learn to choose these actions, and the errors may be passed on through the targets used to train the critic. This is called overestimation bias and can lead to poor decisions.

In this project, we ask two questions. First, does layer normalization (LN) change this bias? Second, does a lower bias lead to better performance? We run DDPG and TD3 on LunarLander, with and without layer normalization in the actor and critic networks. To measure the bias, we compare the critic’s Q-values with Monte Carlo returns obtained by following the policy. The bias cannot be set directly, so we answer the second question indirectly: we check whether the changes that lower the bias (adding LN, or using TD3 instead of DDPG) also improve performance, and whether runs with a lower bias tend to perform better.

2 Experimental Setup

2.1 Models and Layer Normalization

In DDPG, we use neural networks for both the actor π_θ and the critic $\widehat{Q}_\phi^{\pi_\theta}$. The actor produces an action, while the critic estimates its expected discounted return. The critic is trained by minimizing

$$L = \frac{1}{N} \sum_i \left(y_i - \widehat{Q}_\phi^{\pi_\theta}(s_i, a_i) \right)^2,$$

with, for a non-terminal transition,

$$y_i = r_{i+1} + \gamma \widehat{Q}_{\phi'}^{\pi_\theta}(s_{i+1}, \pi_\theta(s_{i+1})).$$

The actor is updated using the critic’s action gradient $\nabla_a \widehat{Q}_\phi^{\pi_\theta}(s_t, a_t)$, evaluated at $a_t = \pi_\theta(s_t)$. As in the course lab, there is no target actor, exploration adds Gaussian noise ($\sigma_{\text{expl}} = 0.1$) to the action, and we bootstrap when an episode is cut by the time limit.

TD3 modifies DDPG in three ways. It trains two critics $\widehat{Q}_{\phi_1}$ and $\widehat{Q}_{\phi_2}$ towards the common target

$$y_i = r_{i+1} + \gamma \min_{j=1,2} \widehat{Q}_{\phi_j'}(s_{i+1}, \pi_{\theta'}(s_{i+1})) + \epsilon, \quad \epsilon = \text{clip}(\mathcal{N}(0, \sigma), -c, c),$$

so that the target uses the more pessimistic of the two estimates; it adds clipped noise to the target action; and it updates the actor and the target networks only once every $K = 2$ critic updates.

For each hidden layer $\ell \in \{1, 2\}$, let $h_{\ell-1}$ denote its input. With layer normalization, the layer computes

$$z_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \text{ReLU}(\text{LN}_\ell(z_\ell)).$$

Thus, normalization acts on the output of the linear transformation, before ReLU.

For a layer with d_ℓ units, we compute, separately for each sample, $\mu_\ell = \frac{1}{d_\ell} \sum_{j=1}^{d_\ell} z_{\ell,j}$ and $v_\ell = \frac{1}{d_\ell} \sum_{j=1}^{d_\ell} (z_{\ell,j} - \mu_\ell)^2$.

The normalization operation is

$$\text{LN}_\ell(z_\ell)_j = g_{\ell,j} \frac{z_{\ell,j} - \mu_\ell}{\sqrt{v_\ell + \varepsilon}} + \beta_{\ell,j}.$$

Here, $\varepsilon = 10^{-5}$, and $g_{\ell,j}$ and $\beta_{\ell,j}$ are learned scale and shift parameters, initialized to 1 and 0, respectively.

This operation is applied independently in both hidden layers of the actor and every critic. Each layer in each network has its own normalization parameters. No normalization is applied to the output layers.

2.2 Monte Carlo Estimation of Bias

Every 25,000 steps, we run 10 episodes and compare the critic’s prediction with the Monte Carlo return from the same state-action pair (s_t, a_t) . The policy π_θ is fixed during evaluation, with no exploration noise. The observed return is

$$G_t^{\text{MC}} = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1},$$

where T is the end of the recorded episode and $\gamma = 0.98$, as in training. When an episode is cut by the 1000-step limit, G_t^{MC} misses the rest of the episode, so we drop its last 194 steps (where the missing weight γ^{T-t} exceeds 0.02); this changes the bias by less than 0.5. Over the M retained pairs, we estimate the bias as

$$\hat{b} = \frac{1}{M} \sum_{i=1}^M \left(\hat{Q}_\phi^{\pi_\theta}(s_i, a_i) - G_i^{\text{MC}} \right).$$

Positive values indicate average overestimation while negative values indicate average underestimation. For TD3, the bias is computed using the predictions of the first critic. The bias of a run is the average of its measurements after 100,000 steps, which excludes the early phase where the critic is still untrained.

2.3 Hyperparameter Selection

We explored three values of the target update coefficient, $\tau \in \{0.001, 0.005, 0.05\}$, and two hidden-layer widths, 64 and 256 units. This gives six configurations per algorithm, each trained with three seeds.

We initially ranked configurations using the last evaluation at 350,000 transitions. After inspecting the learning curves, we changed the criterion to the average of the last ten evaluations, from 305,000 to 350,000 transitions. Each evaluation contains ten episodes. We first average these evaluations within each training run, then average the resulting scores across the three seeds.

This criterion measures performance maintained during the last part of training, rather than performance at a single checkpoint. Since the criterion was revised after inspecting the results, we treat this search as exploratory.

2.4 Final Models and Training Protocol

The selected configurations are reported in Table 1. For each algorithm, we compare training with and without layer normalization, keeping all other hyperparameters fixed. Each condition is run with 20 seeds, giving 80 training runs in total. All training and analysis scripts are available in our GitHub repository.¹

Parameter	DDPG	TD3
Hidden layers	(64, 64)	(64, 64)
Actor / critic learning rate	10^{-3}	10^{-3}
Discount factor γ	0.98	0.98
Target-update coefficient τ	0.05	0.05
Batch size	64	64
Critic updates per actor update	1	2
Training steps	350 000	350 000
Seeds per condition	20	20

Table 1: Final training configurations on LunarLanderContinuous-v3.

2.5 Statistical Analysis

The performance of a run is the average of its last ten evaluations. We fix six comparisons in advance: LN versus no LN for each algorithm, and TD3 versus DDPG without LN, each on bias and on performance. We test them with Welch’s t -test, which does not assume equal variances, with a Bonferroni correction over the six tests. We report the difference of means with a 95% bootstrap confidence interval and the probability that a run of A beats a run of B; we also show performance profiles. The link between bias and performance is measured with Spearman’s rank correlation, with a Bonferroni correction over its five tests (four configurations and all runs pooled). In the figures, variability is shown as the interval between the 10th and 90th percentiles of the runs.

¹<https://github.com/Youssef75ensae/ddpg-td3-layernorm-lunarlander>

3 Results

3.1 Effect of LayerNorm on the bias

Figure 1 shows that, without LayerNorm, the critic of DDPG strongly overestimates: by about 60 points early in training and about 20 points at the end. With LayerNorm, its bias decreases faster, so its curve stays lower between 50k and 250k steps; the two curves then meet around 300k steps. LayerNorm therefore mainly speeds up the correction of the bias rather than changing its final level.

For DDPG, this corresponds to reduced overestimation. For TD3, the average bias is already close to zero without LayerNorm and becomes more negative with it, indicating increased underestimation. Thus, a lower signed bias does not necessarily mean more accurate value estimates.

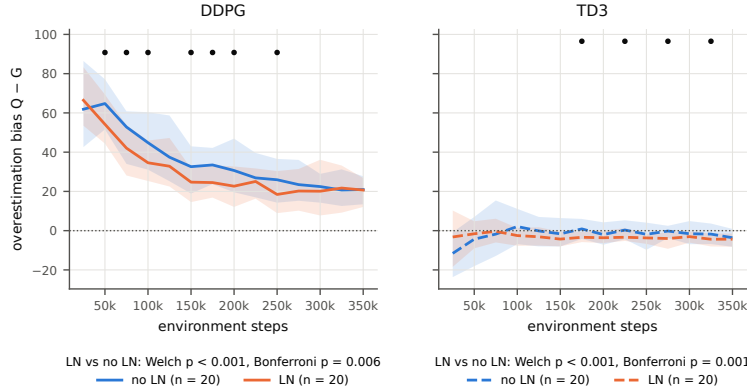


Figure 1: **Signed bias of the critic along training** ($\hat{b} = \hat{Q} - G^{\text{MC}}$), without LN (blue) and with LN (orange). Lines: mean over 20 seeds; shaded areas: 10th–90th percentile interval; black dots: steps where LN and no LN differ (Welch’s test, $p < 0.05$, uncorrected).

3.2 Effect of bias on performance

Figure 2 shows that DDPG begins to plateau towards the end of training, while TD3 is still improving. Our comparison is therefore limited to the 350 000-step budget and may not reflect performance after convergence.

We detect no statistically significant effect of LayerNorm on end-of-training performance, measured over the last ten evaluations (Bonferroni-adjusted $p = 1.000$ for both algorithms). This does not establish that LayerNorm has no effect, but indicates that our experiment does not provide sufficient evidence of a difference.

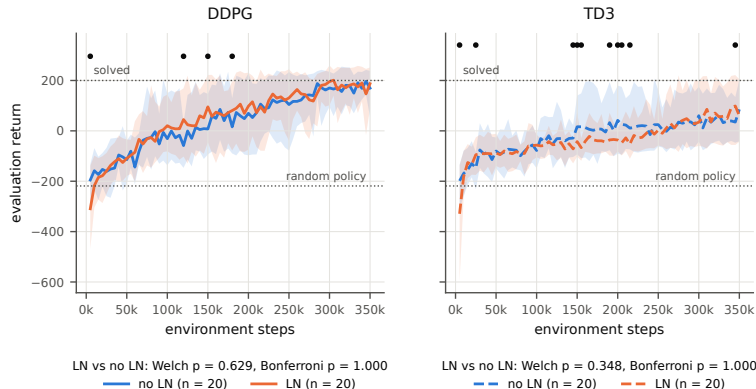


Figure 2: **Learning curves for DDPG and TD3 with and without LayerNorm** (mean return of 10 deterministic episodes every 5 000 steps). Lines: mean over 20 seeds; shaded areas: 10th–90th percentile interval; black dots: steps where LN and no LN differ (Welch’s test, $p < 0.05$, uncorrected); dotted lines: random policy and the 200 threshold. Takeaway: the curves with and without LN overlap, and TD3 is still improving at 350 000 steps.

LayerNorm lowers the bias of both algorithms without changing their performance. The comparison between the two algorithms points in the same direction: without LayerNorm, TD3 overestimates 29.9 points less than DDPG, yet its final performance is 129 points lower. TD3 also differs from DDPG in other ways and has not converged, so this gap cannot be attributed to the bias alone, but removing the bias clearly did not help here.

Figure 3.(a) shows the fraction of runs whose end-of-training performance exceeds a given threshold. At a return of 200, this fraction is 30% for DDPG, both with and without LayerNorm, compared with 0% for TD3 without LayerNorm and 5% with it. DDPG therefore achieves better performance under the tested configurations and training budget.

Figure 3.(b) shows the relationship between bias and performance, with one point per training run. The pooled correlation is positive (Spearman’s $\rho = 0.57$), but may mainly reflect the differences between algorithms: DDPG has both higher performance and higher overestimation than TD3. Within each configuration, no correlation remains significant after Bonferroni correction. With 20 runs per configuration, however, only strong correlations ($|\rho|$ above about 0.68) would be detected with 80% probability, so weak links cannot be excluded. These results do not establish a causal effect of overestimation on performance.

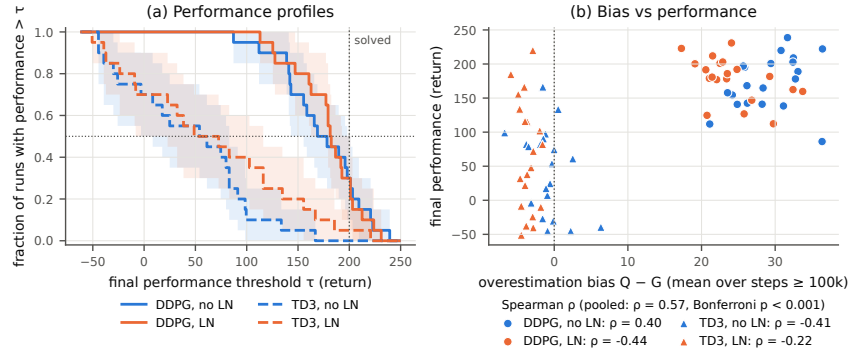


Figure 3: **Distribution of final performance, and its relation to the bias.** (a) Fraction of runs whose final performance exceeds a threshold (horizontal axis), with 95% bootstrap bands. (b) Final performance against bias, one point per run; Spearman correlations of each configuration in the legend. Takeaway: DDPG dominates TD3, and the positive pooled correlation only reflects the gap between the two algorithms.

4 Conclusion

We compared DDPG and TD3 on **LunarLanderContinuous-v3**, with and without LayerNorm in the actor and critic networks. After selecting the network width and target-update coefficient through a small grid search, we trained the four conditions using 20 seeds each and 350 000 environment steps. We measured the bias by comparing critic predictions with discounted Monte Carlo returns from the same state-action pairs.

For DDPG, LayerNorm reduced the mean bias from 29.0 to 24.1. This reduction is significant after Bonferroni correction ($p = 0.006$), although the estimates remain positively biased on average. For TD3, the mean bias changed from -0.9 to -3.6 , also significantly ($p = 0.001$). Here, LayerNorm increases underestimation. The distinction matters: decreasing the signed bias does not necessarily bring the estimates closer to the observed returns.

The performance results are less conclusive. Mean performance was higher with LayerNorm: 178.4 versus 172.9 for DDPG, and 65.8 versus 44.0 for TD3. However, neither difference was statistically significant after Bonferroni correction ($p = 1.000$ for both algorithms). These results leave the effect of LayerNorm on performance uncertain.

We also found no significant correlation between bias and performance within any configuration after correction. The positive pooled correlation ($\rho = 0.57$) may mainly reflect the differences between algorithms: DDPG performs better despite having more overestimation. We therefore cannot infer that overestimation improves performance, or that reducing it necessarily improves learning.

Our conclusions concern the tested configurations and the 350 000-step budget. In particular, TD3 was still improving near the end of training. Longer runs would help assess whether these results persist. Since LayerNorm was applied jointly to the actor and critics, separate experiments would also be needed to identify their respective contributions.